# Sensitivity workflow

SWAT+ has no built-in sensitivity analysis. Sensitivity is run by an external tool that writes `calibration.cal`, launches the SWAT+ executable, reads an objective value from the simulation output, and repeats. The parameter list and bounds are taken from the same `cal_parms.cal` used by hard calibration.

## Pattern

1. Pick the parameters to test. Pull their absolute ranges from `cal_parms.cal` or override them with a tighter calibration range. Each parameter is one dimension of the sample space.
2. Generate a sample matrix using a sensitivity method (Sobol, FAST, Morris, LH-OAT, …). Each row of the matrix is one parameter vector.
3. For each row: write `calibration.cal` with one `pctchg` (or `absval`, `abschg`) record per parameter, run SWAT+, read the chosen objective (NSE, KGE, percent bias, total flow, …) from the output files, and record it.
4. Feed the sample matrix and the model performance vector into the sensitivity method to compute indices: first-order, total-order, second-order interaction, or elementary effects depending on the method.

The expensive step is the simulation loop. A sample size in the hundreds to low thousands is common.

## sensitivityAPI

sensitivityAPI is a command-line tool that implements steps 2 and 4 of the pattern. It operates on a SWAT+ TxtInOut directory: it reads a parameter CSV (`par_data.stb`), writes a sample matrix (`par_sample.stb`), and after the user has run the simulations and collected performance values into `perf_report.stb`, computes sensitivity indices.

Supported methods:

| Method | Sampling | Indices |
|---|---|---|
| `FAST` | frequency-based | S1, ST |
| `sobol` | Saltelli quasi-random | S1, ST, S2 |
| `RBD_FAST` | Latin Hypercube | S1 |
| `DMIM` | Latin Hypercube | S1 (delta) |
| `lhoat` | LH-OAT hybrid | S1 (elementary effects) |

Usage:

```
sensitivityAPI generate_sample sobol /path/to/txtinout 1024
# run the simulations here, fill perf_report.stb
sensitivityAPI analyse_sensitivity sobol /path/to/txtinout
```

The orchestration (writing `calibration.cal` per row, launching SWAT+, parsing the output) is done by a driver such as sptAPI or spToolboX. `sensitivityAPI` is only the sampler and the post-processor.

## Roll your own

Any language with a numerical library can do the same. The minimum loop:

1. Parse `cal_parms.cal` to get bounds for the parameters you care about.

2. Generate `N` parameter vectors with a Python (`SALib`), R (`sensitivity`), or in-house sampler.
3. For each vector, overwrite `calibration.cal` with the corresponding `pctchg` records and call the SWAT+ binary with the TxtInOut directory as the working directory.
4. Read the metric from `channel_sd_day.txt` (or whichever output your objective uses), record it.
5. Hand the matrix and the vector to your sensitivity routine.

## Related pages

- Parameter change files. Format of `cal_parms.cal` and `calibration.cal`.
- Hard calibration. The mechanism each sensitivity run drives, one parameter vector at a time.